Isaac Sim AprilTag Detection and Handling

Project Documentation — Perception-driven object handling for a UR5e in Isaac Sim over ROS 2

Robot: Universal Robots UR5e + Robotiq 2F-85 gripper | Sim: NVIDIA Isaac Sim 6.0.1 (Windows) | ROS 2 Jazzy (WSL2, Ubuntu 24.04) | Package: `ur5e_isaac_control`

Date: 2026-07-13

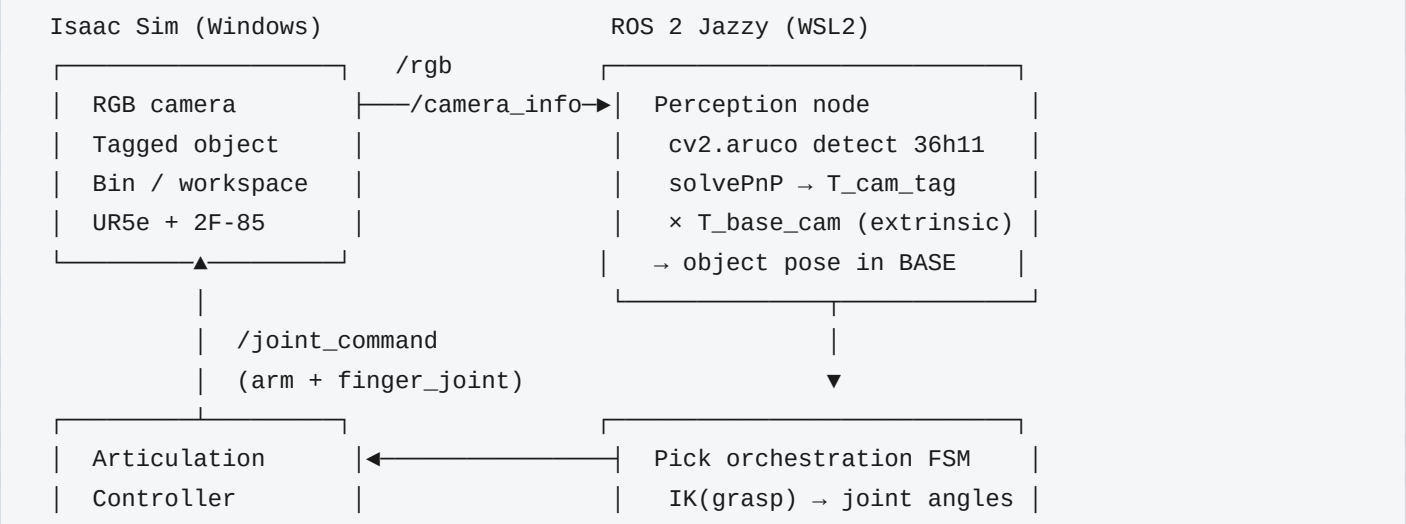
1. Overview

This project detects an **AprilTag**-marked object in an Isaac Sim scene using a simulated RGB camera, estimates the object's full 6-DOF pose, and drives a UR5e arm to pick and handle it. It is the perception-driven evolution of a joint-control base already proven end-to-end (commanding the arm and gripper from ROS 2 into the Isaac Sim viewport).

The perception approach is deliberately lightweight and dependency-free: AprilTag fiducials give a robust, exact 6-DOF pose with only OpenCV — no GPU inference, no Docker, and no external model downloads — which suits the Isaac-Sim-on-Windows / ROS 2-in-WSL2 setup.

> **Status legend.** **BUILT & VALIDATED** = implemented and tested in this repository. **PLANNED** = designed, scheduled for implementation against the live scene.

2. System Architecture



```
└──────────────────┘                 │   approach→grasp→lift→place │
      /joint_states ──────────────────►  (current pose feedback)   │
                                       └─────────────────────────────┘
```

The ROS 2 ↔ Isaac Sim link runs over a unicast-loopback Fast-DDS profile that works around a WSL2 mirrored-mode multicast-discovery bug; full detail is in `SETUP_NOTES.md`.

3. Components

3.1 Kinematics — `kinematics.py` [BUILT & VALIDATED]

UR5e forward/inverse kinematics built on `ikpy` and the official `ur_description` URDF (generated via `xacro`, `ur_type:=ur5e`). The kinematic chain runs `base_link → … → tool0` (the ROS-Industrial tool frame); the unused `ft_frame` sensor branch is trimmed at load so the chain terminates cleanly at `tool0`, and only the six revolute arm joints are active.

Method	Description
`fk(q6)`	6 joint angles (rad) → 4×4 TCP pose in `base_link`
`ik(target_pose, initial_q6)`	4×4 TCP target → 6 joint angles (rad)
`pose_error(q6, target)`	(position error [m], orientation error [rad])
`tcp_z_offset` (ctor)	Metres from `tool0` to the grasp point along tool +Z (gripper length)

Validation: 200 random reachable configurations, round-trip `IK(FK(q)) → q'`: **max position error 0.0000 mm, max orientation error 0.0000°, 0 failures.** FK sanity: reach at zero pose = 0.852 m (matches UR5e 0.85 m spec); TCP offset verified to shift exactly along tool +Z.

3.2 AprilTag pose — `apriltag_pose.py` [BUILT & VALIDATED]

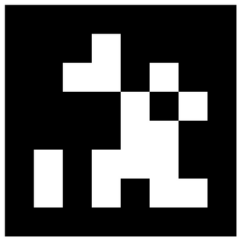
Detects 36h11 AprilTags with OpenCV's `cv2.aruco.ArucoDetector` (sub-pixel corner refinement) and recovers each tag's 6-DOF pose with `cv2.solvePnP` using `SOLVEPNP_IPPE_SQUARE` (optimal for planar squares). Since `estimatePoseSingleMarkers` was removed in OpenCV 4.7+, pose is computed from the tag's known metric corner geometry — the current standard method.

Method	Description

`detect(image, K, dist)`	→ list of {id, corners, `T_cam_tag`, rvec, tvec}
`pose_from_corners(pts, K, dist)`	4 tag corners → (`T_cam_tag`, rvec, tvec); pose math, decoupled for testing
`draw_detections(...)`	Debug overlay: tag outline, axes, id

Pose convention: `T_cam_tag` expresses the tag frame in the camera optical frame (OpenCV convention: X right, Y down, Z out of the lens). The tag frame sits at the tag centre, Z out of the face.

Validation: detection confirmed on a rendered 36h11 marker; pose math over 50 known ground-truth poses (projected corners → `pose_from_corners` → recovered pose): **max position error 0.0002 mm, max orientation error 0.0007°.**



Generated tag: 36h11, id 0 (white quiet-zone border included — required for detection). Tags 0–3 in `apriltags/`.

3.3 Interactive joint controller GUI — `joint_controller_gui.py` [BUILT & VALIDATED]

A Tk slider panel (one slider per joint, live radians/degrees readout) that streams the pose to `/joint_command` at 30 Hz and snaps to the arm's real state on startup. Used as the learning/tuning tool for hand-finding grasp poses. Confirmed moving the arm live in the Isaac Sim viewport.

3.4 Generated AprilTags [BUILT]

Four print-ready 36h11 tags (ids 0–3, 1040×1040 px, 15% white quiet-zone border) in `apriltags/`. The **black-square side length** (inner 800 px region) is the metric `tag_size` the pose estimator needs.

4. Isaac Sim scene setup [PLANNED — user task]

Recommended: a **fixed camera** (eye-to-hand) looking down at the workspace, so the camera→base

transform is static and set once.

- **Object:** a ~5 cm Cube with RigidBody + Collider; one tag PNG applied to the top face. Record the black-square side in metres as `tag_size`.
- **Bin:** a box/tray as the place target.
- **Camera:** a `Camera` prim above the workspace, fixed resolution (e.g. 1280×720).
- **ROS 2 publishing:** a **ROS2 Camera Helper** node (type `rgb`, plus `camera_info`) in the Action Graph, wired to the camera, topics `/rgb` and `/camera_info`.
- Press **Play** (do a Stop→Play cycle after any graph edit).

The camera extrinsic (`T_base_cam`) will be read directly from the USD stage for exactness, rather than measured by hand.

5. Perception & picking [PLANNED]

Node	Role
Perception node	Subscribe `/rgb` + `/camera_info`; run `apriltag_pose`; transform `T_cam_tag` by the static extrinsic `T_base_cam`; publish the object pose in the robot base frame.
Pick orchestration FSM	On a detected tag: compute a top-down grasp pose, solve IK, then execute approach → descend → close gripper (`finger_joint`) → lift → move to bin → open gripper → home.

The gripper is already confirmed commandable over `/joint_command` (`finger_joint`: 0.0 = open, ~0.6–0.8 = closed), enabling a real grasp rather than a mimic.

6. Environment & dependencies

Item	Detail
OpenCV	4.12 (ships `cv2.aruco` AprilTag families — zero extra install)
IK	`ikpy` 4.0.0 (user site) + `ur_description` URDF
ROS 2	Jazzy; `rmw_fastrtps_cpp`; unicast-loopback DDS profile (see `SETUP_NOTES.md`)
Vision support	NumPy, SciPy, `cv_bridge`, `image_geometry` (all present)

7. Validation summary

Check	Result
IK round-trip (200 poses)	**0.0000 mm / 0.0000°, 0 failures**
FK reach at zero pose	**0.852 m (matches UR5e spec)**
AprilTag detection (36h11)	**id recovered correctly**
AprilTag pose math (50 poses)	**0.0002 mm / 0.0007°**
Arm motion in viewport	**confirmed via joint controller GUI**
Gripper command	**finger_joint 0.0 → 0.60 confirmed**

8. Roadmap

- **Done:** joint controller, FK/IK, AprilTag detection + pose, tag assets.
- **Next:** Isaac Sim scene (camera + tagged object); read camera extrinsic from stage.
- **Then:** perception ROS 2 node; pick orchestration FSM; end-to-end pick-and-place of the tagged object.

9. Asset index

Path	Contents
`src/ur5e_isaac_control/ur5e_isaac_control/kinematics.py`	UR5e FK/IK
`src/ur5e_isaac_control/ur5e_isaac_control/apriltag_pose.py`	AprilTag detect + pose
`src/ur5e_isaac_control/ur5e_isaac_control/joint_controller_gui.py`	Interactive joint slider GUI
`src/ur5e_isaac_control/urdf/ur5e.urdf`	UR5e URDF (for IK)
`apriltags/apriltag_36h11_id{0..3}.png`	Print-ready AprilTags
`SETUP_NOTES.md`	Isaac Sim ↔ ROS 2 bridge

setup